MIAE 215 · Programming for Mechanical & Industrial Engineers

Midterm 1-Page Memory Aide & Rapid Exam Survival Blueprint

Concordia University · Gina Cody School of Engineering and Computer Science

1. Data Types, Memory Sizes & Literal Syntax

Type	Bytes	Range / Precision	Typical Use	Concordia Exam Trap
`bool`	**1**	`true` (1) or `false` (0)	Flags, loop conditions	Non-zero ints evaluate to `true` (`-1` is `true` !)
`char`	**1**	-128 to 127 (ASCII: `'0'` =48, `'A'` =65, `'a'` =97)	Single characters, menu choices	`'a' + 1` evaluates to `98` (promoted to `int`)
`int`	**4**	$\approx -2.14 \times 10^9$ to $+2.14 \times 10^9$	Counters, indices, whole numbers	**Integer Division Truncation**: `5 / 2 == 2`
`long long`	**8**	$\approx \pm 9.22 \times 10^{18}$	Large integer factorials, timestamps	Suffix `LL` needed for large literals
`float`	**4**	$\approx 10^{-38}$ to 10^{38} (6–7 sig digits)	Fast graphics, memory conservation	Never compare with `==` due to round-off
`double`	**8**	$\approx 10^{-308}$ to 10^{308} (15–17 sig digits)	Engineering simulations, high-precision	Default floating literal (`3.14` is `double`)

> ⚠️ **The #1 Exam Calculation Trap — Integer Division**:
>
> ```
> int a = 7, b = 2;
> double res1 = a / b;                          // ❌ res1 = 3.0 (integer division truncates first!)
> double res2 = static_cast<double>(a) / b;     // ✔ res2 = 3.5 (explicit cast promotes operation)
> double res3 = 1.0 / 2;                        // ✔ res3 = 0.5 (floating literal forces float division)
> ```

2. Operator Precedence & Evaluation Rules

1. **Grouping & Postfix**: `()` `[]` `x++` `x--`

(Left-to-Right)

2. **Unary**: `++x` `--x` `+` `-` `!` `&` (address-of) `*` (dereference) `sizeof`
   (Right-to-Left)
3. **Multiplicative**: `*` `/` `%`
   (Left-to-Right)

   → Modulo `%` requires integer operands only!
4. **Additive**: `+` `-`
   (Left-to-Right)
5. **Relational**: `<` `<=` `>` `>=`
   (Left-to-Right)
6. **Equality**: `==` `!=`
   (Left-to-Right)
7. **Logical AND**: `&&`
   (Left-to-Right, Short-circuit)
8. **Logical OR**: `||`
   (Left-to-Right, Short-circuit)
9. **Conditional (Ternary)**: `cond ? expr1 : expr2`
   (Right-to-Left)
10. **Assignment**: `=` `+=` `-=` `*=` `/=` `%=`
    (Right-to-Left)

> 🧠 **Prefix vs. Postfix Evaluation**:
>
> - `y = x++;` ⟹ Assign current `x` to `y`, **THEN** increment `x`.
> - `y = ++x;` ⟹ Increment `x` **FIRST**, then assign new value to `y`.
> - **Short-Circuit Law**: In `(A && B)`, if `A == false`, `B` is **never executed**. In `(A || B)`, if `A == true`, `B` is **never executed**.

---

3. Control Flow & Dr. Gordon's Trace Table Protocol

Selection Structures

- **Equality vs. Assignment Trap**: `if (x = 5)` assigns `5` to `x`, evaluates to `true`, and overwrites memory! Always write `if (x == 5)`.
- **Switch Case Rules**: Expression must be integral (`int`, `char`, `bool`). Floating types (`double`) and `string` are illegal. Without `break`, execution falls through into subsequent cases!

Signature Question 2: Dr. Gordon's Table Tracing Protocol

When asked to predict output or trace variables through nested loops:

1. **Draw Columns**: One column per variable (`i` , `j` , `sum` , `a` , `b` , array elements) plus an **Output** column.
2. **Line-by-Line Execution**: Record a value **only when that variable changes**. Leave cell blank if unchanged.
3. **Cout Trigger**: Every time `cout` executes, write the characters **immediately** into the Output column.
4. **Array Boundary / Garbage (`G`) Trap**: For `double A[4]` , valid indices are `0, 1, 2, 3` . If a loop finishes with `i = 4` and calls `cout << A[i]` , it accesses out-of-bounds memory → **Output is Garbage (`G`)**.

```
// Midterm Classic: Loop Trace with Postfix & Out-of-Bounds
int arr[3] = {10, 20, 30};
int i = 0, sum = 0;
while (i < 3) {
    sum += arr[i++]; // Uses arr[0], then increments i to 1!
}
cout << "sum=" << sum << ", i=" << i << endl; // Prints: sum=60, i=3
// cout << arr[i];                          // ⚠️ arr[3] is OUT-OF-BOUNDS -> Garbage (G)!
```

---

4. Arrays, Functions & Parameter Passing

1D & 2D Array Architecture

- **Declaration & Initialization**: `int A[5] = {1, 2};` ⟹ remaining elements zero-initialized: `{1, 2, 0, 0, 0}` .
- **Passing Arrays to Functions**: Arrays **decay to pointers** (`int a[]` is actually `int* a`). Changes inside the function modify the original caller array! `sizeof(a)` inside a function returns pointer size (8 bytes), NOT array size.

Pass-by-Value vs. Pass-by-Reference

```
void swapVal(int a, int b) { int t = a; a = b; b = t; }      // ❌ Local copies only! Caller unchanged.
void swapRef(int &a, int &b) { int t = a; a = b; b = t; }    // ✔ References alias original variables!
void printVec(const vector<int> &v);                         // ✔ Const reference: Zero copy, read-only.
```

---

5. Flowchart & Flowgorithm Standard Symbols

- **Oval / Pill (Terminator)**: Start / End / Return.
- **Parallelogram (I/O)**: `cin >> val;` / `cout << res;` .
- **Rectangle (Process)**: Computation, variable assignments (`x = a + b;` , `int sum = 0;`).
- **Diamond (Decision)**: Conditional branching (`x > 0 ?`) with True and False branches.

---

6. High-Frequency Midterm Algorithms Checklist

1. **Dual Array Equality Check**:

```
bool isEqual = true;
for (int i = 0; i < 5; i++) {
    if (A1[i] != A2[i]) { isEqual = false; break; }
}
if (isEqual) cout << "Arrays are identical.\n";
else cout << "Arrays differ.\n";
```

2. **Min/Max Array Search**: Always initialize `int maxVal = A[0];` (never initialize to `0` because array could contain all negative numbers!).
3. **Formatted Console I/O (`<iomanip>`)**:
    - `cout << fixed << setprecision(2) << val;` (persistent 2 decimal places).
    - `cout << setw(8) << val;` (affects ONLY the next single output value).